

APPENDIX A – EXPLAINABILITY GRAPHS - FE JAVA

A.1 Feature Importance

Figure 1 shows the feature importance for detecting FE in Java projects. Features were selected using the ANOVA technique, considering 30% of the available attributes. To compute feature importance, we used the *LossFunctionChange* method in CatBoost's *get_feature_importance* function. This type of feature importance can be applied to any model, but it is particularly useful for classifiers, where other importance types might provide misleading results. For each feature, the value represents the difference between the models loss with and without that feature. The most relevant metrics for detecting FE in Java projects were AFTD_type and MAXNESTING_method.

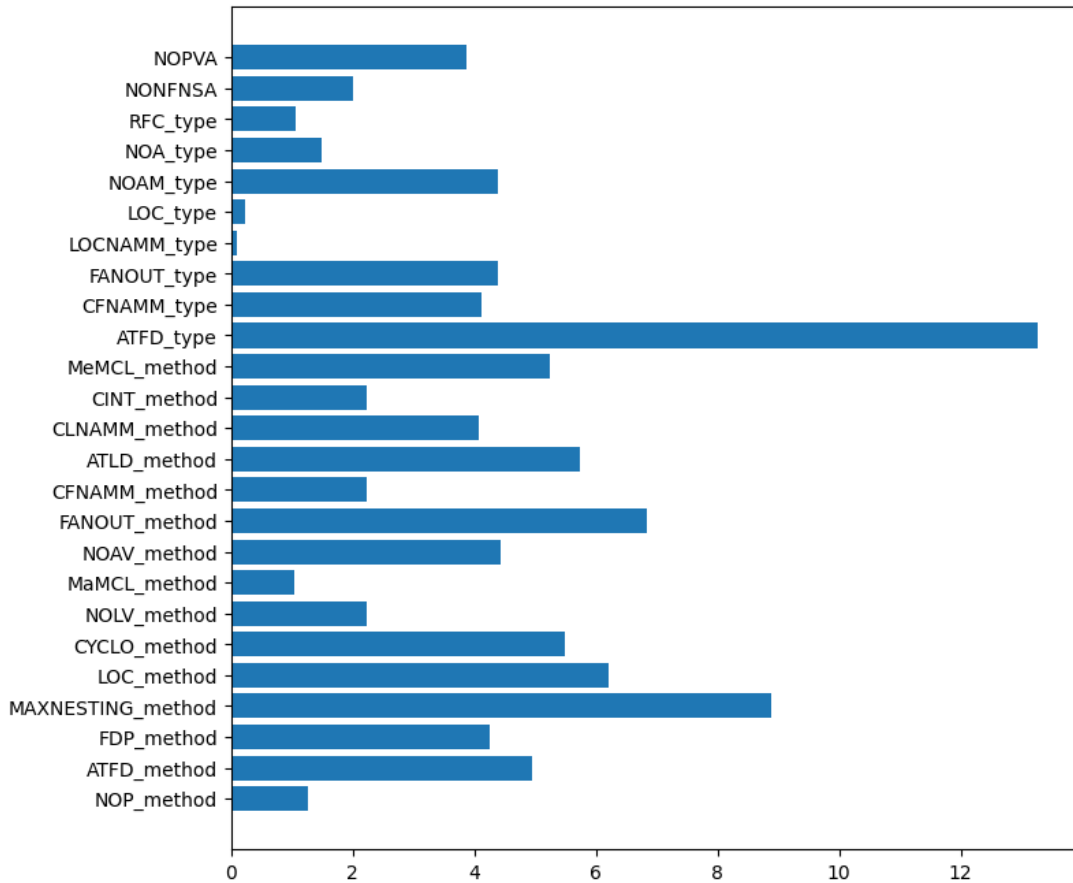


Figure 1 – Feature Importance for FE Detection in Java

Figure 2 shows the feature importance for severity assessment of FE in Java projects. Features were selected using the ANOVA technique, considering 25% of the available attributes. Feature importance was calculated using a Random Forest classifier by computing the total impurity reduction from the Gini criterion across the models trees.

The most relevant metrics for severity assessment of FE in Java projects were AFTD_type and MAXNESTING_method.

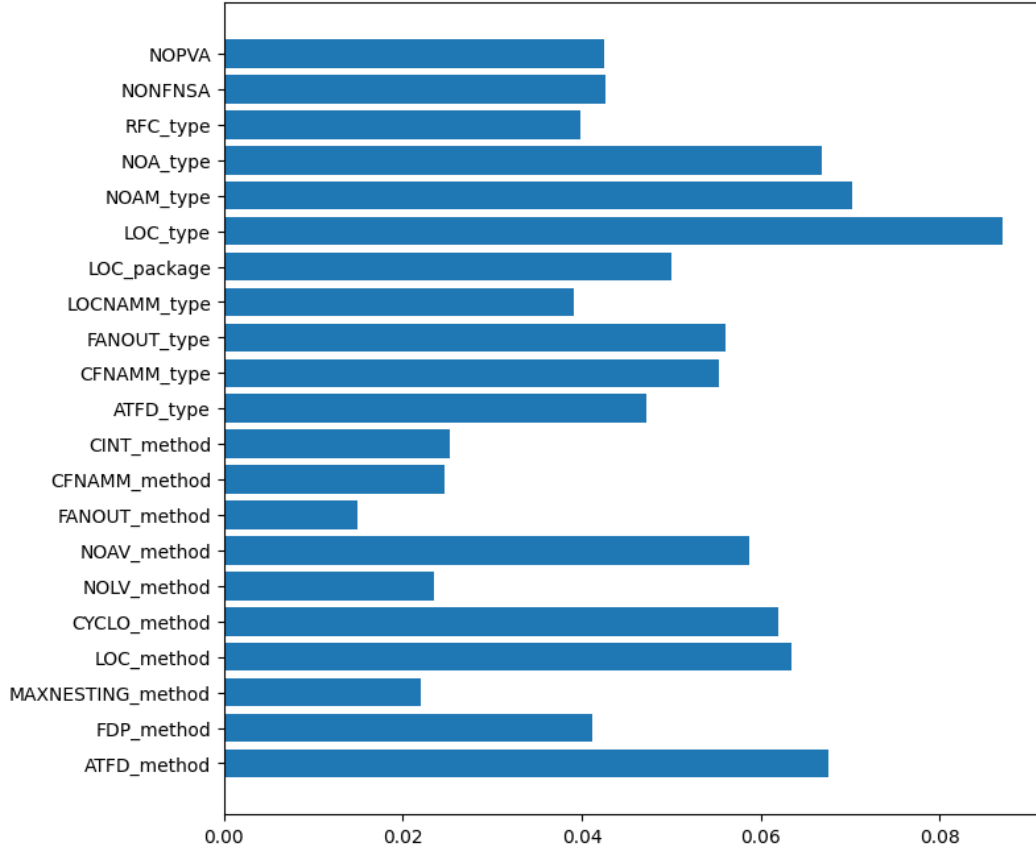


Figura 2 – Feature Importance for FE Severity Assessment in Java

A.2 SHAP

SHAP is a game-theoretic approach to explain the output of any machine learning model. It connects optimal credit allocation with local explanations using classic Shapley values from game theory and related extensions. SHAP is particularly useful for understanding the impact of each variable at both the global and local levels, across any model type (1). To derive global insights from local importances, the SHAP summary plot was used to evaluate the impact of features on the output of the CatBoost model for FE detection, as shown in Figura 3. In SHAP summary plots, each row represents a variable, and each dot represents an instance in the dataset. The dot color indicates the attribute value: blue for low values and red for high values. The horizontal position reflects the SHAP value, i.e., the attribute's effect: further to the right indicates a greater positive contribution of that attribute to the instance (1).

In the plot above (Figura 3), the attributes AFTD_type, AFTD_method, and FDP_method contribute differently depending on their value to the models response: i)

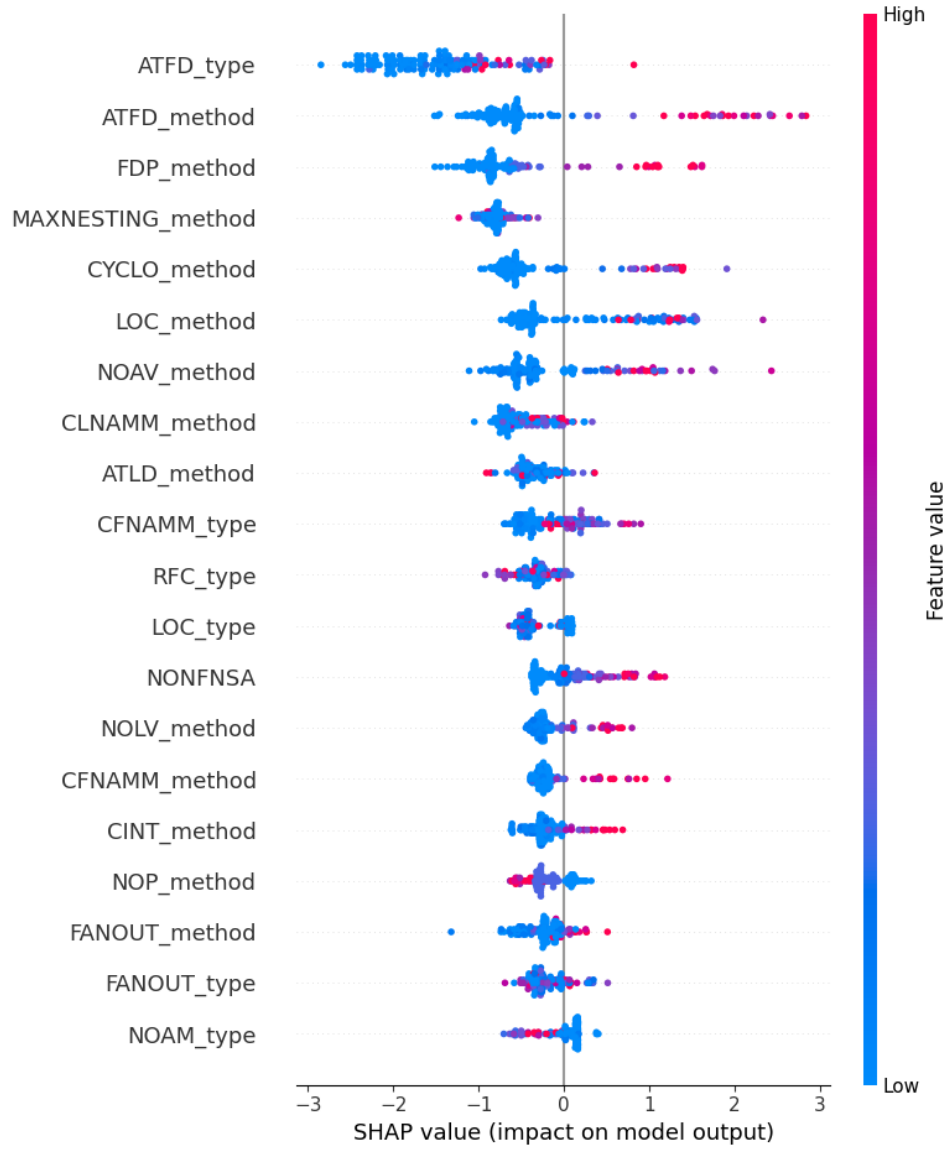


Figura 3 – Impact on the Output of the FE Detection Model in Java - SHAP

for lower values (blue), they contribute negatively, and ii) for higher values (red), they contribute positively. In general, variables with greater horizontal dispersion of SHAP values tend to be more important. This is because a wide variation in SHAP values indicates that the variable affects the prediction across different instances, suggesting a greater influence on the models decision-making process¹.

A.3 LIME

LIME is an algorithm that explains local model predictions by fitting a simple interpretable model (such as linear regression) near the point of interest. It is useful for understanding specific predictions and for complex models such as Random Forests and

¹ <<https://medium.com/big-data-blog/shap-o-que-%C3%A9-e-por-que-usar-6b01d37ae592>>

Boosting algorithms (2). Therefore, this algorithm was used to interpret the classifiers for code smell severity assessment. For FE severity in Java projects, we used a Random Forest-based model with standardized data.

Figura 4 presents the LIME plot for a non-severe FE instance, which in this case has a 64% probability of being correctly classified by the severity assessment model. This instance has standardized values of $NOA_type > 0.19$, $-0.23 < ATFD_method \leq 0.64$, and $-0.39 < CINT_method \leq 1.18$, which increase the model's confidence. However, the values $ATFD_type > 0.34$ and $RFC_type > 0.00$ negatively impact the prediction probability.

For a regular FE severity instance (Figura 5), the prediction probability reached 81%. The standardized values of $-0.13 < NOAV_method \leq 1.28$, $-0.22 < NOLV_method \leq 1.43$, and $-0.47 < NOA_type \leq -0.24$ were relevant for increasing this confidence. Conversely, $CINT_method > 1.18$ and $FDP_method > 1.56$ decreased the models confidence.

For a severe FE instance (Figura 6), the values $ATFD_method > 0.64$, $NOAV_method > 1.28$, $ATFD_type > 0.34$, and $RFC_type > 0.00$ were decisive in reaching a 76% prediction probability for this higher severity level.

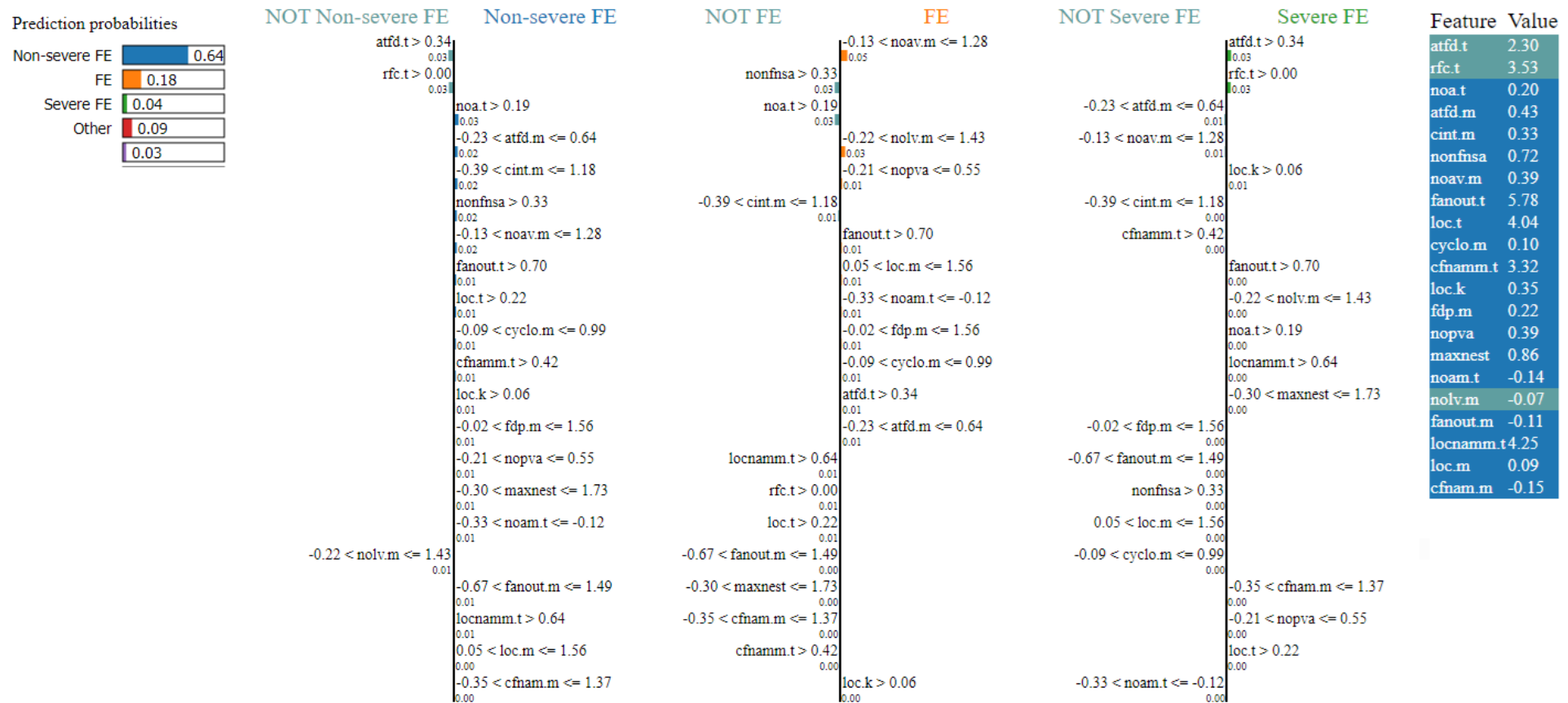


Figura 4 – LIME Explanation of a Non-Severe FE Instance in Java

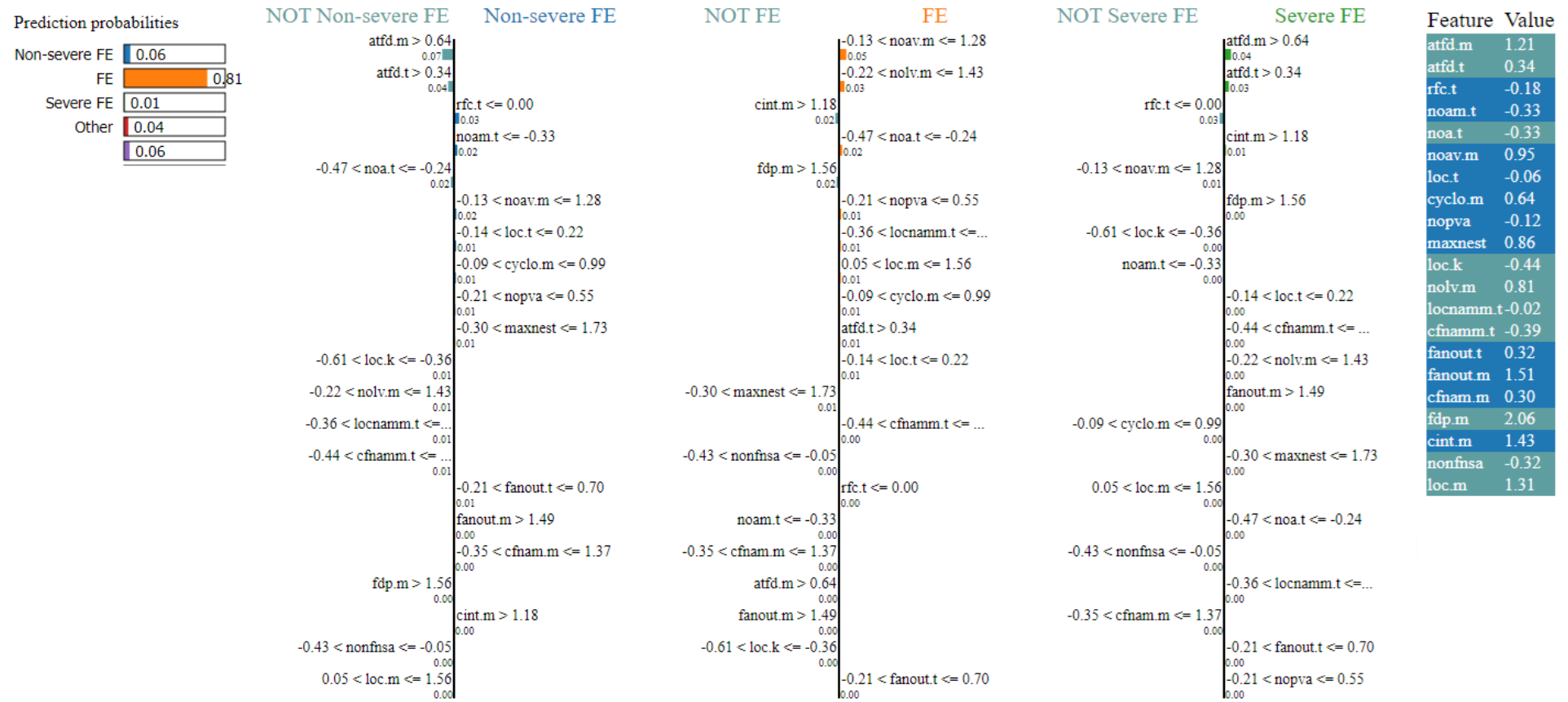


Figure 5 – LIME Explanation of a Regular FE Severity Instance in Java

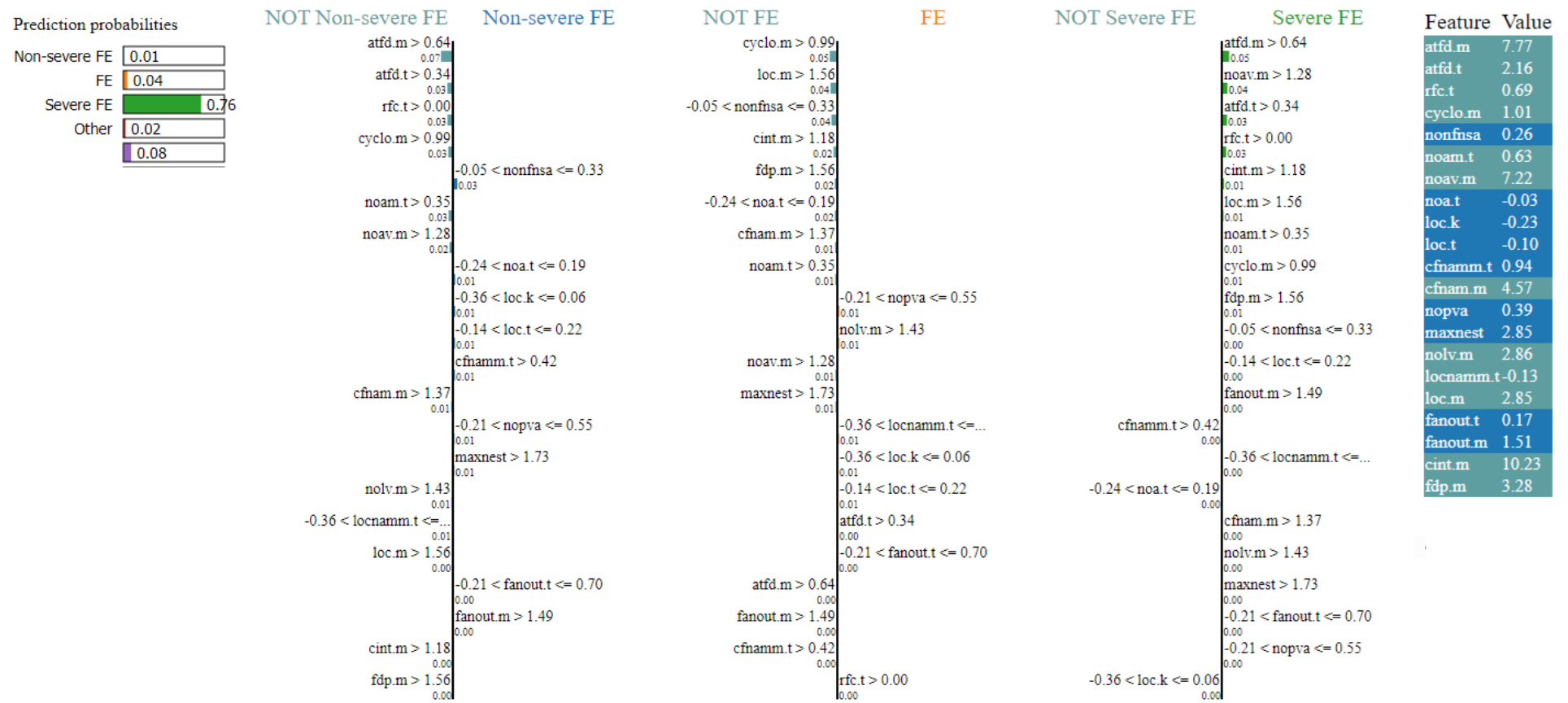


Figura 6 – LIME Explanation of a Severe FE Instance in Java

APPENDIX B – EXPLAINABILITY GRAPHS - LM JAVA

B.1 Feature Importance

To compute feature importance for both the detection of LM in Java projects and the evaluation of its severity, the default value of the *importance_type* parameter in XGB, “*weight*”, was used, which measures importance by the number of times a feature appears in decision tree nodes.

Figure 7 shows the feature importance for detecting LM in Java projects. A total of 25 features were selected using the ANOVA technique, corresponding to 30% of the available features. MAXNESTING_method, FANOUT_method, and ATFD_type were the most important metrics for detecting LM in Java projects.

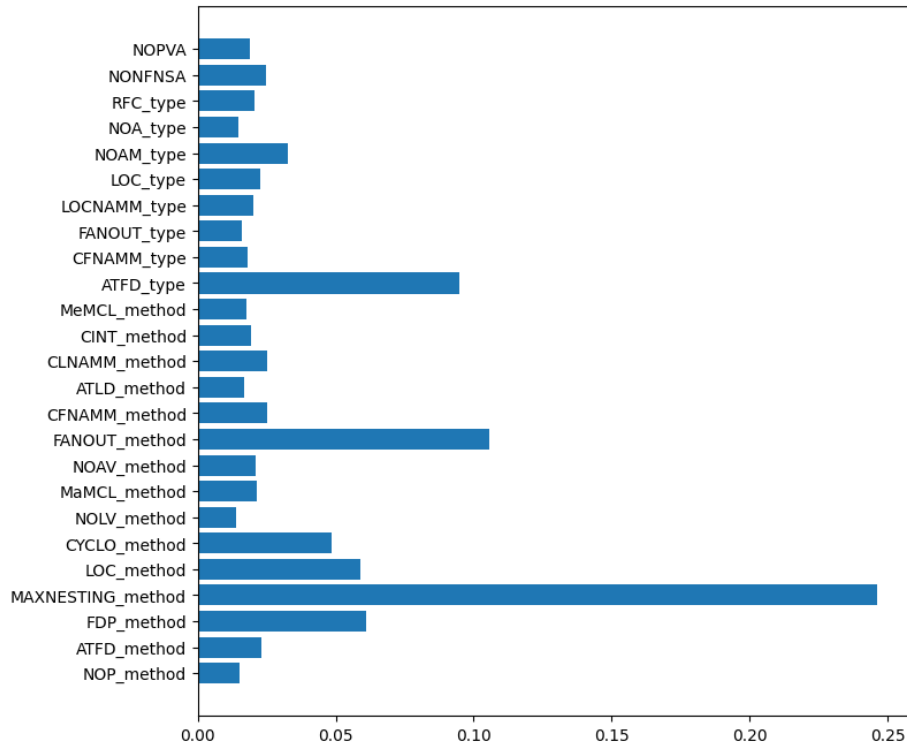


Figure 7 – Feature Importance for LM Detection in Java

Figure 8 shows the feature importance for severity evaluation of LM in Java projects. Features were selected using the Chi-Square technique, corresponding to 25% of the available features. The most relevant metrics for LM severity evaluation in Java were LOC_method, WMCNAMM_type, CYCLO_method, and ATFD_method.

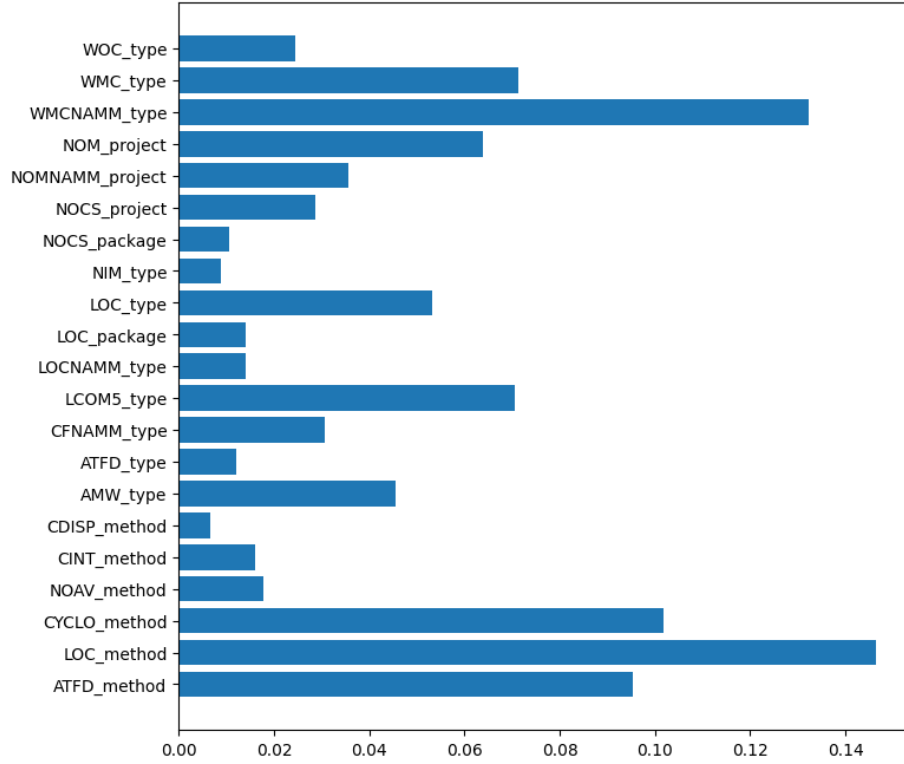


Figura 8 – Feature Importance for LM Severity Evaluation in Java

B.2 SHAP

Figura 9 presents the SHAP summary plot showing the impact of features on the output of the XGB model for LM detection. MAXNESTING_method, LOC_method, FANOUT_method, and FDP_method were the most impactful features on the model output. For these features, lower values (blue) negatively influenced the model output, while higher values (red) positively influenced it.

B.3 LIME

For LM severity evaluation in Java projects, a model based on XGB without data scaling was used. Figura 10 shows the LIME plot for a non-severe LM instance, which achieved an 82% probability of being correctly classified by the severity evaluation model. In this instance, the values of the features $NOM_project > 4624.00$ and $0.00 < CINT_method \leq 10.26$ positively influenced the prediction probability. Meanwhile, the following feature values: $5.00 < NOAV_method \leq 20.89$ and $WMCNAMM_type \leq 21.00$ negatively influenced the prediction.

Figura 11 shows a 100% prediction probability for the LM severity evaluation. The following feature values were important for this maximum prediction probability: $CYCLO_method > 16.17$, $NOAV_method > 20.89$, $9.52 < CFNAMM_type \leq 28.00$,

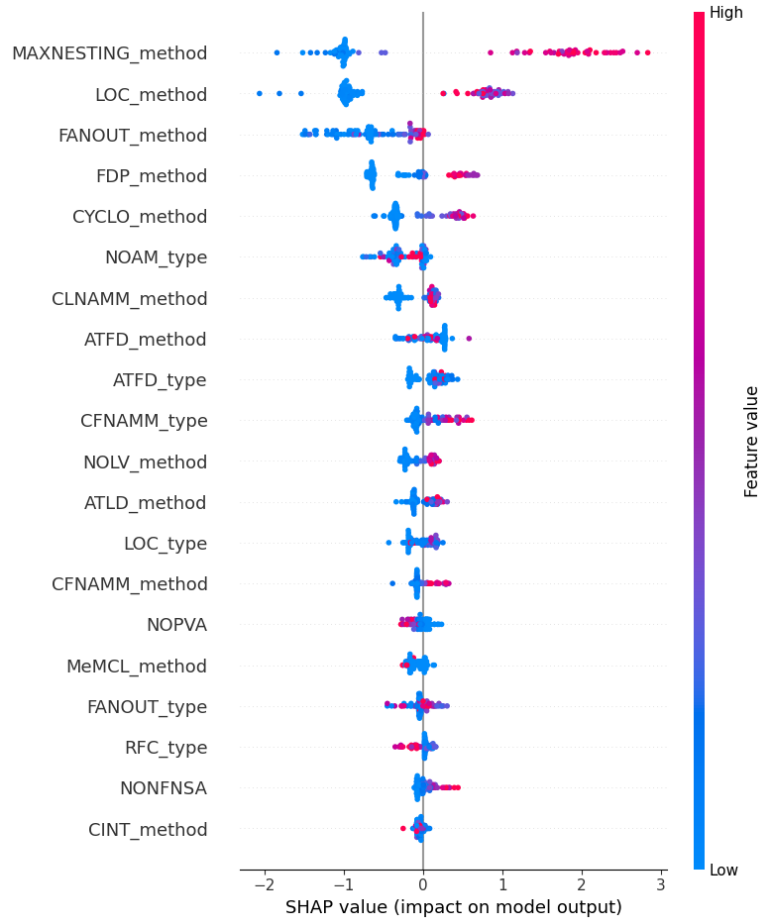


Figura 9 – Impact of Features on the Output of the Java LM Detection Model - SHAP

$28.00 < LOC_method \leq 104.00$, $LOCNAMM_type > 148.26$, $NIM_type \leq 2.20$, and $WOC_type \leq 0.26$. It is worth noting that although some feature values – $0.00 < CINT_method \leq 10.26$ and $LCOM5_type > 0.94$ – contributed negatively, they were not enough to reduce the prediction score for this severity instance.

In the case of a severe LM instance, detailed in Figura 12, the feature values $AMW_type \leq 1.47$, $WMCNAMM_type \leq 6.00$, $CFNAMM_type \leq 1.98$, and $LCOM5_type > 0.93$ increased the probability of correct classification. For this specific case, the prediction probability was 84%. This prediction probability was negatively affected by the values of the following features: $NOC'S_package > 48.00$, $LOC_package > 7478.00$, and $0.11 < WOC_type \leq 0.40$.

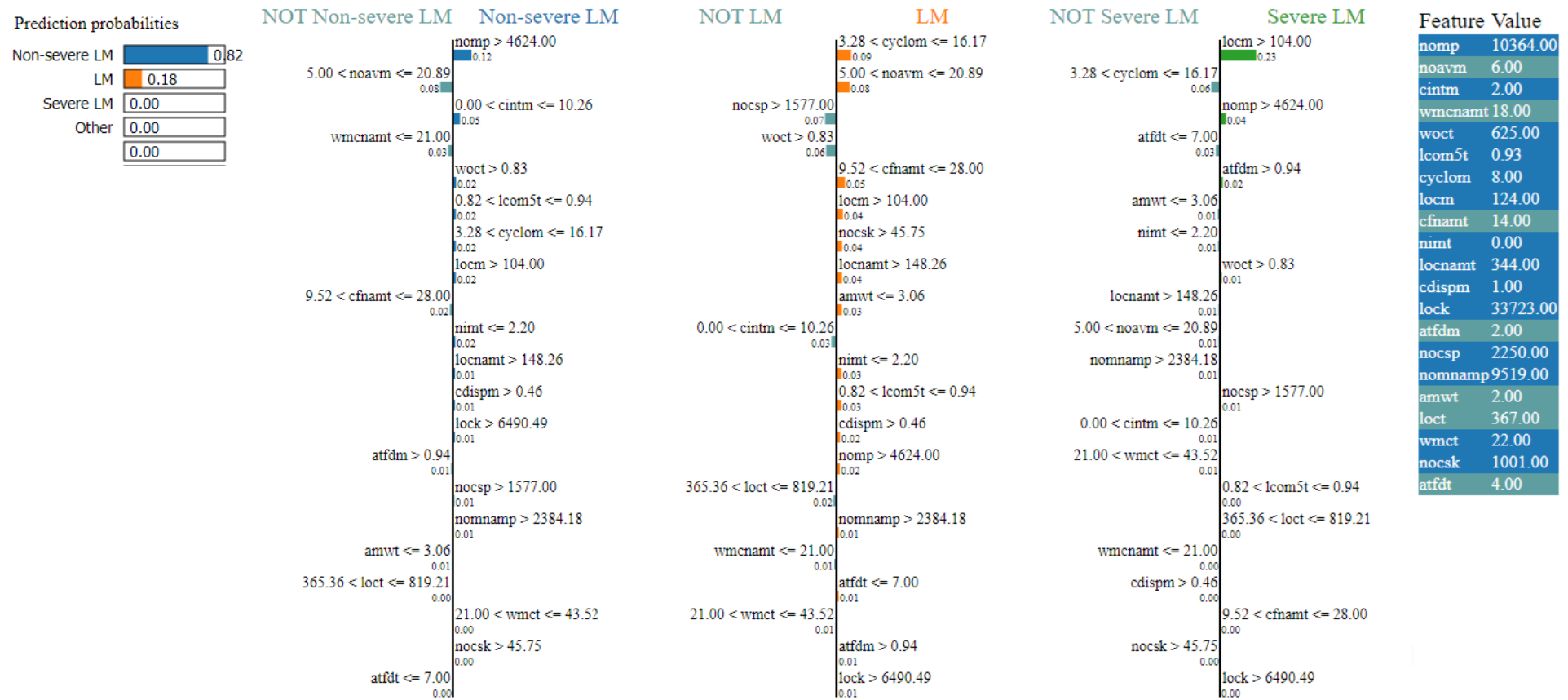


Figure 10 – LIME Explanation of a Non-Severe LM Instance in Java

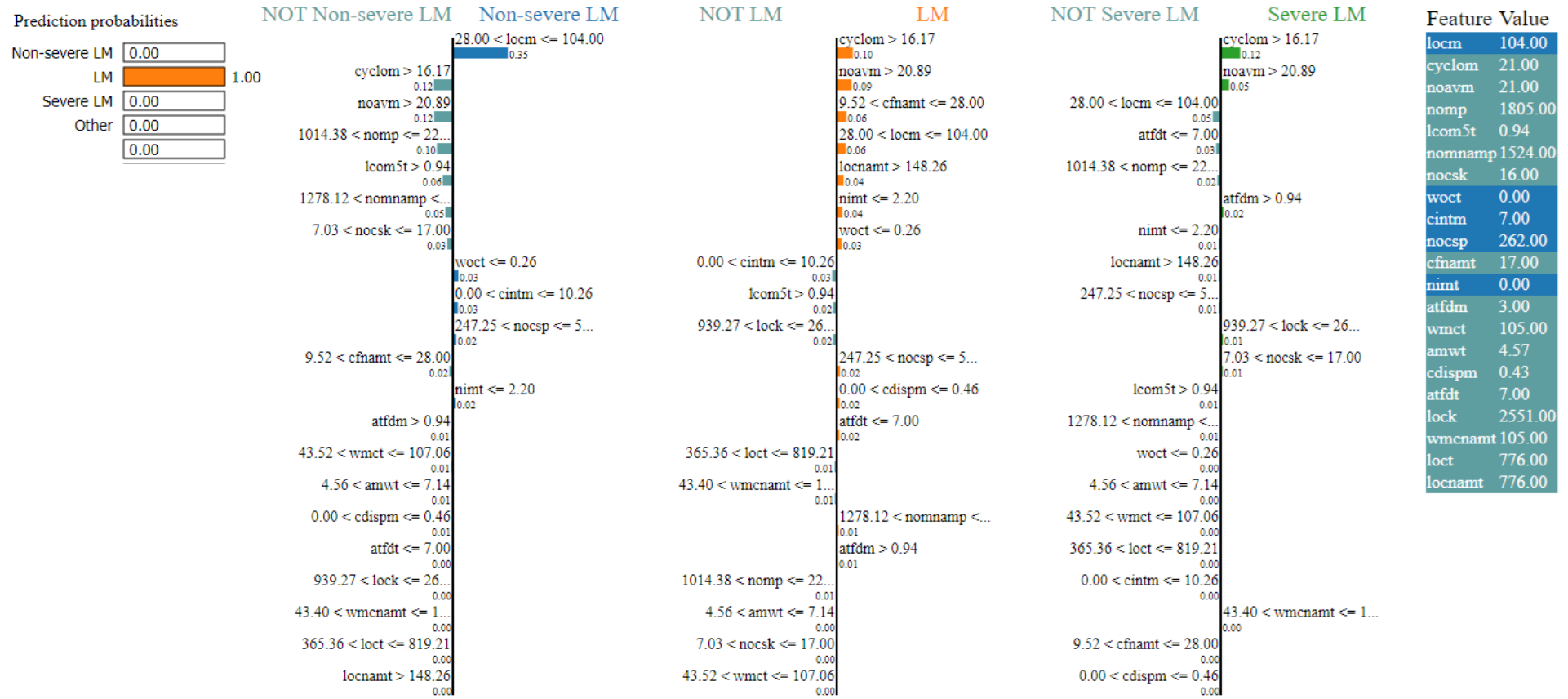


Figura 11 – LIME Explanation of an LM Severity Instance in Java

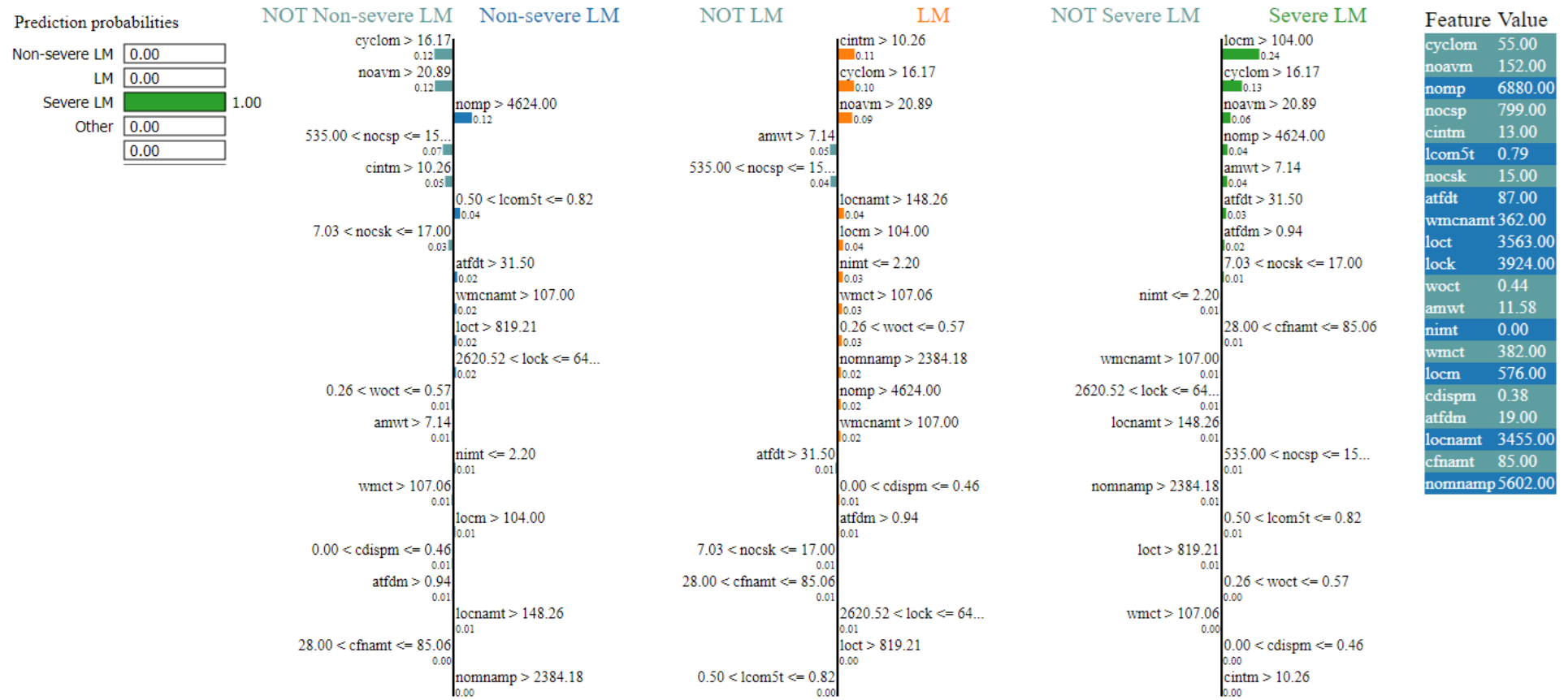


Figure 12 – LIME Explanation of a Severe LM Instance in Java

APPENDIX C – EXPLAINABILITY GRAPHS - JAVA DC

C.1 Feature Importance

Figura 13 shows the importance of features for the detection of Data Class (DC) in Java projects. A total of 25 features were selected using the Chi-Square technique, corresponding to 30% of the 84 available features. Feature importance was calculated using the default *importance_type* parameter in XGBoost, "weight", which measures importance based on the number of times a feature appears in decision tree nodes. The metrics that stood out the most for DC detection in Java projects were LOC_method, LOC_type, and NOCS_project.

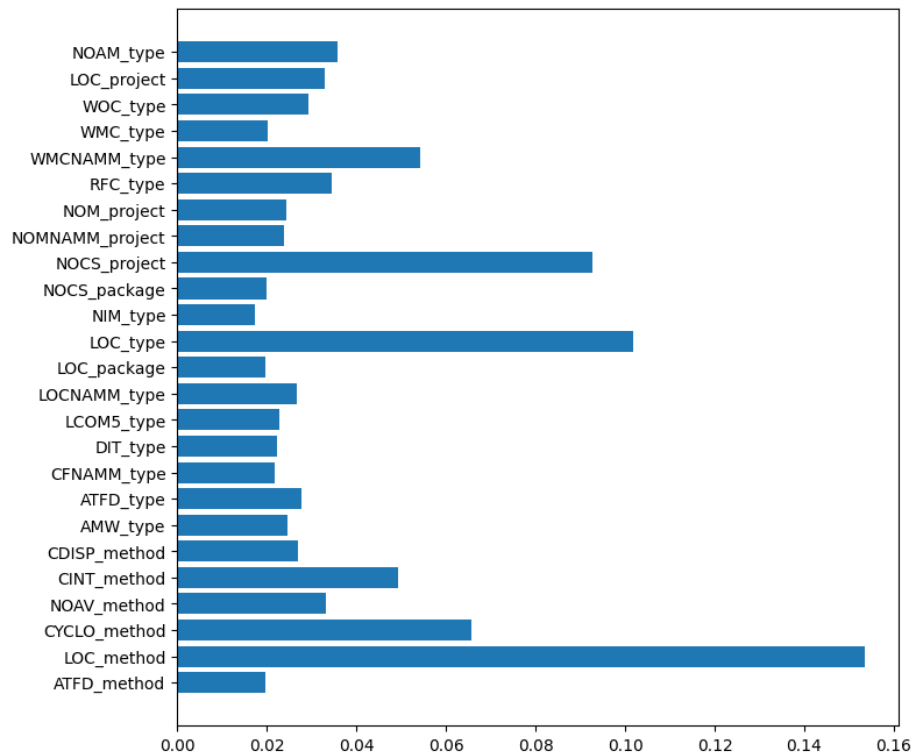


Figura 13 – Feature Importance for DC Detection in Java

Figura 14 shows the importance of features for severity assessment of DC in Java projects. The features were selected using the Chi-Square technique, representing 25% of the available features. The feature importance calculation, based on a Random Forest classifier, was performed by computing the total Gini impurity reduction across the model's trees. The metrics that stood out the most for DC severity assessment in Java projects were WMC_type, LOC_method, WMCNAMM_type, LOC_type, and CYCLO_method.

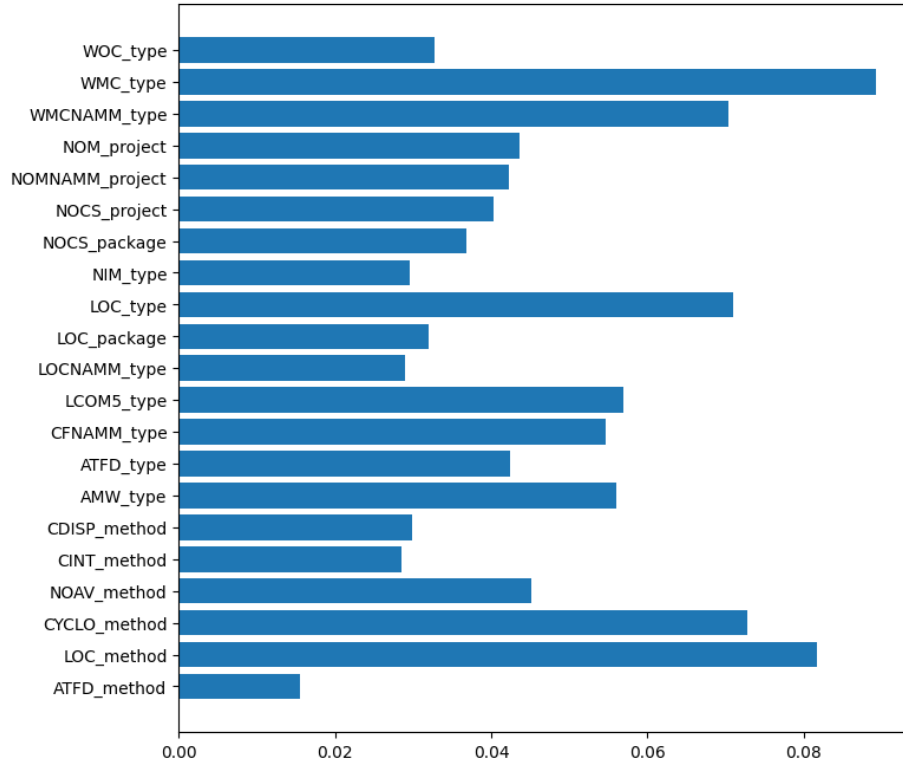


Figure 14 – Feature Importance for DC Severity Assessment in Java

C.2 SHAP

Figure 15 shows the SHAP summary plot illustrating the impact of features on the XGBoost model output for DC detection. NOCS_project, LOC_method, NOAM_project, and LCOM5_type were the features with the greatest impact on the model output. For NOCS_project and LCOM5_type, lower values (blue) negatively impact the prediction, while higher values (red) have a positive effect. Conversely, for LOC_method and NOAM_type, most lower values contribute positively, while most higher values contribute negatively.

C.3 LIME

To assess DC severity in Java projects, a Random Forest-based model was used without data scaling. Figure 16 presents the LIME chart for a non-severe DC instance, which in this case has a 99% probability of being correctly classified by the severity assessment model. This instance has the following features: $WMC_type \leq 12.70$, $LOC_type \leq 107.00$, $WMCNAMM_type \leq 6.00$, $AMW_type \leq 1.47$, $LOC_method \leq 3.00$, $CYCLO_method \leq 1.00$, $CFNAMM_type \leq 1.98$, and $NOAV_method \leq 1.00$, all contributing to this high probability.

For the DC severity instance shown in Figure 17, the model's confidence is 87%,

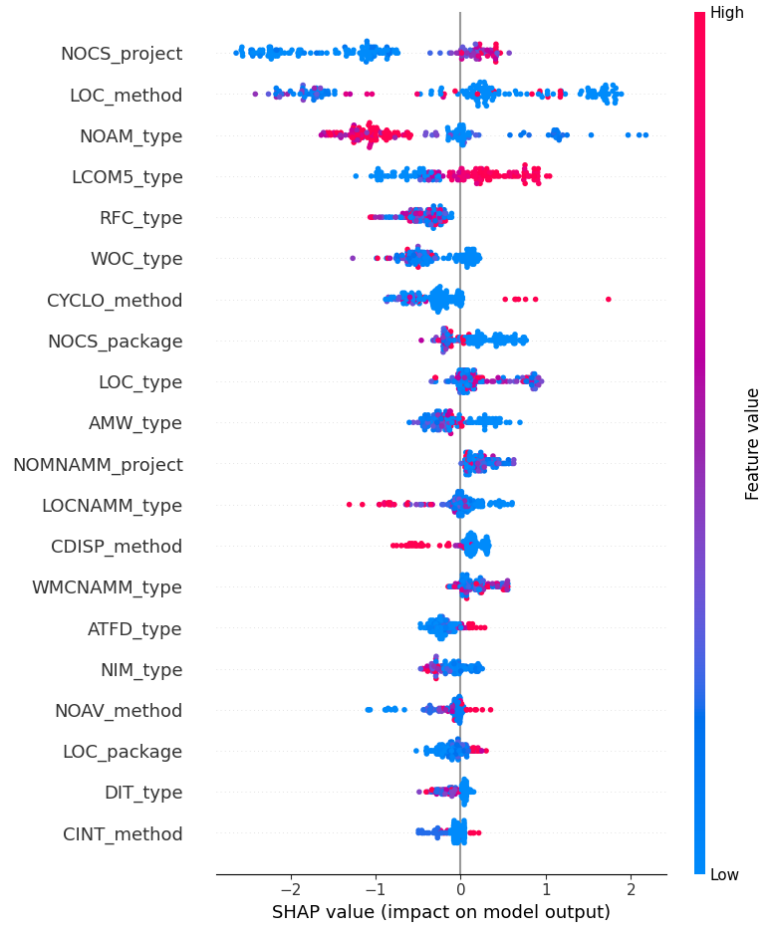


Figura 15 – Impact on Model Output for Java DC Detection - SHAP

influenced by feature values such as: $WMCNAMM_type \leq 6.00$, $AMW_type \leq 1.47$, $CFNAMM_type \leq 1.98$, $CYCLO_method \leq 1.00$, $LOC_method \leq 3.00$, $ATFD_type \leq 1.00$, $NOAV_method \leq 1.00$, and $LCOM5_type > 0.85$. On the other hand, the values $LOC_package \leq 1006.71$ and $WMC_type \leq 12.70$ reduce the prediction probability.

In the case of a severe DC instance, detailed in Figura 18, the prediction probability is 84%. Contributing features include: $AMW_type \leq 1.47$, $WMCNAMM_type \leq 6.00$, $CFNAMM_type \leq 1.98$, $LCOM5_type > 0.93$, $CYCLO_method \leq 1.00$, $NOM_project > 5064.00$, $NIM_type \leq 0.25$, $ATFD_type \leq 1.00$, and $LOC_method \leq 3.00$, all increasing the likelihood of a correct classification.

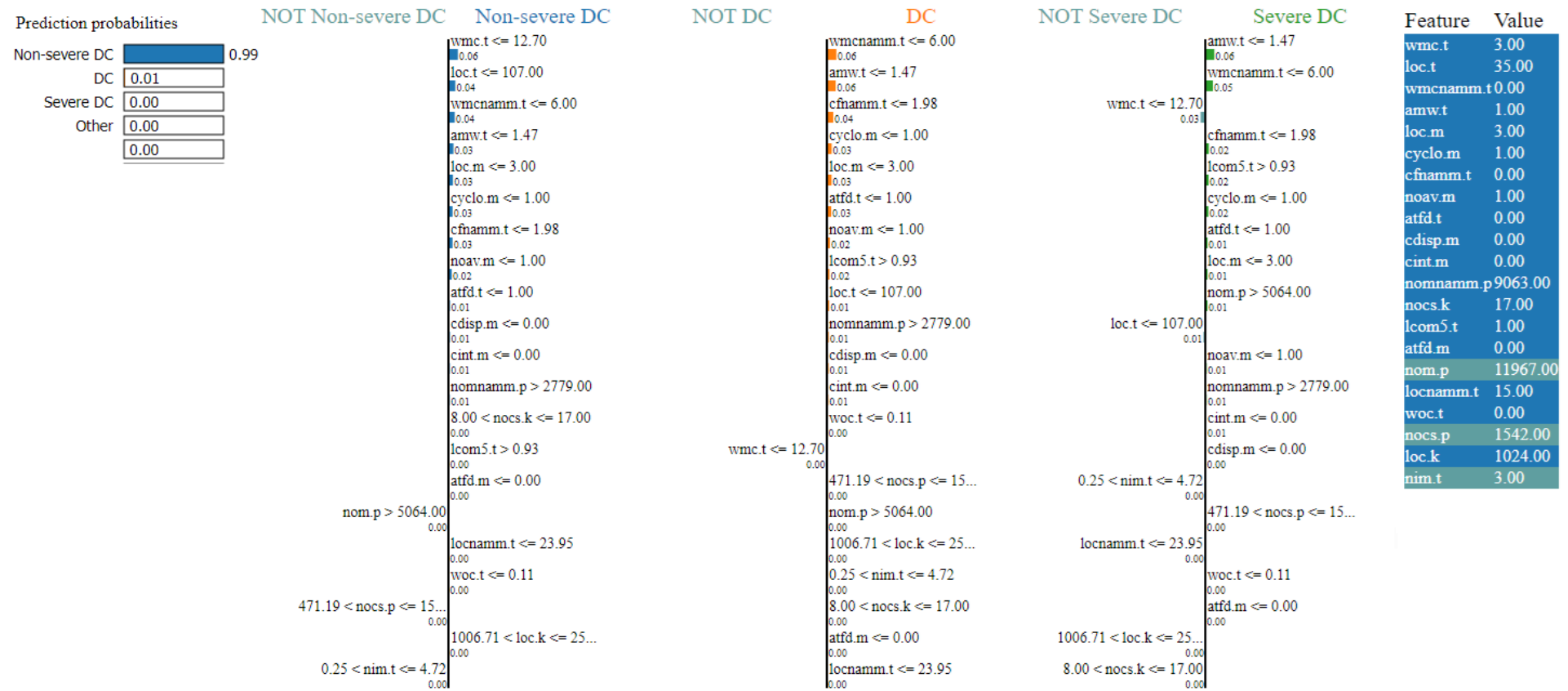


Figura 16 – Details of a Non-Severe Java DC Instance using LIME

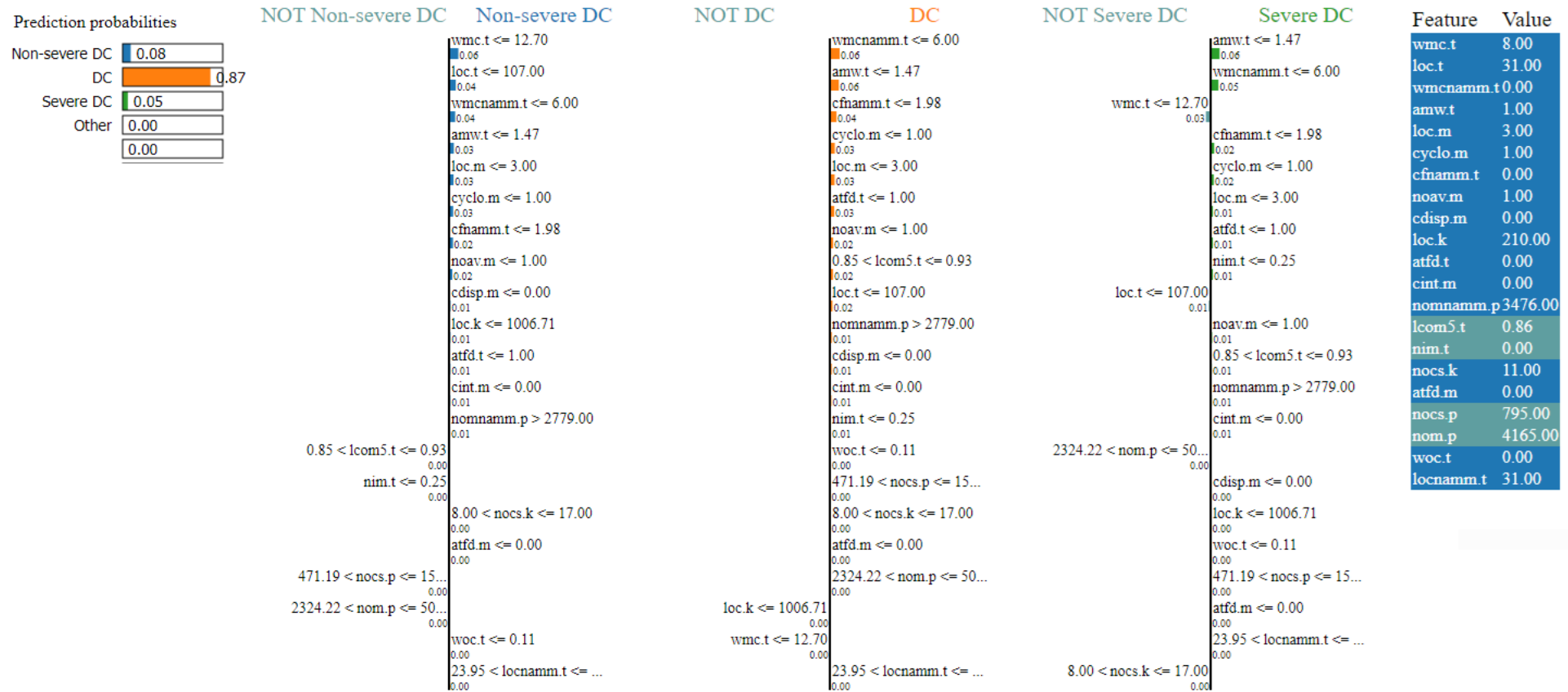


Figura 17 – Details of a Java DC Severity Instance using LIME

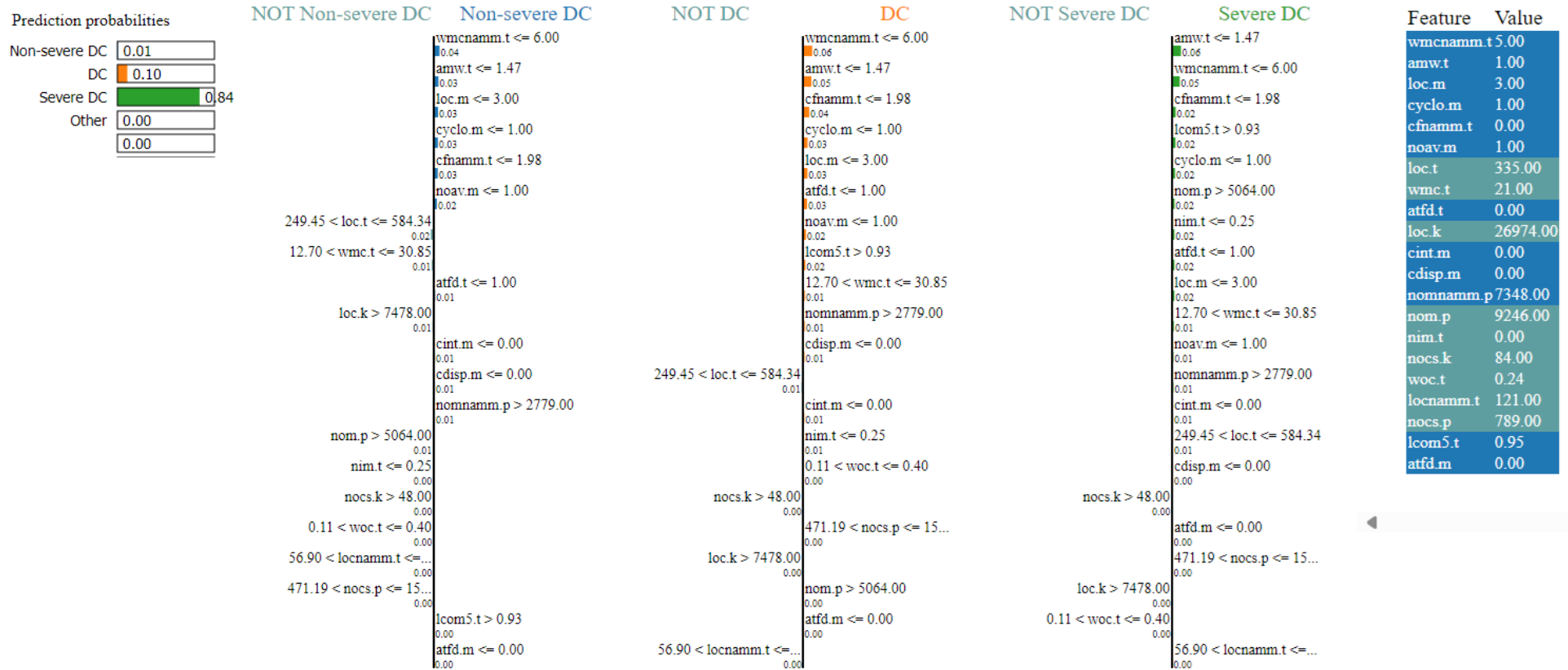


Figura 18 – Details of a Severe Java DC Instance using LIME

APPENDIX D – EXPLAINABILITY GRAPHS GC JAVA

D.1 Feature Importance

To compute feature importance for both GC detection and severity assessment in Java projects, the *LossFunctionChange* approach was used with the *get_feature_importance* method from CB. For each feature, the value represents the difference in the model's loss value with and without that feature.

Figura 19 shows the feature importance for GC detection in Java projects. The selected subset of features corresponds to 20% of the available features. These features were selected using the Chi-Square technique. The metrics with the greatest importance for detecting GC in Java projects were LOC_method, CYCLO_method, CDISP_method, and WMCNAMM_type.

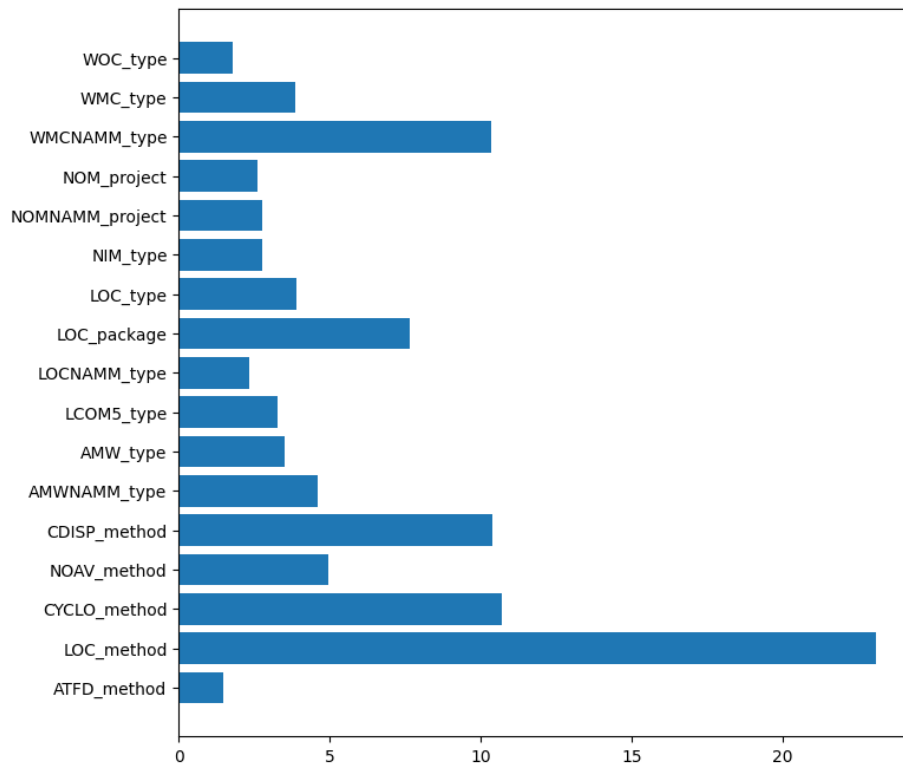


Figura 19 – Feature Importance for GC Detection in Java

Figura 20 shows the feature importance for GC severity assessment in Java projects. The features were selected using the ANOVA technique, corresponding to 30% of the available features. FANOUT_type, MeMCL_method, ATFD_method, and ATFD_type were the metrics that showed the best results in the importance calculation for severity assessment in Java GC.

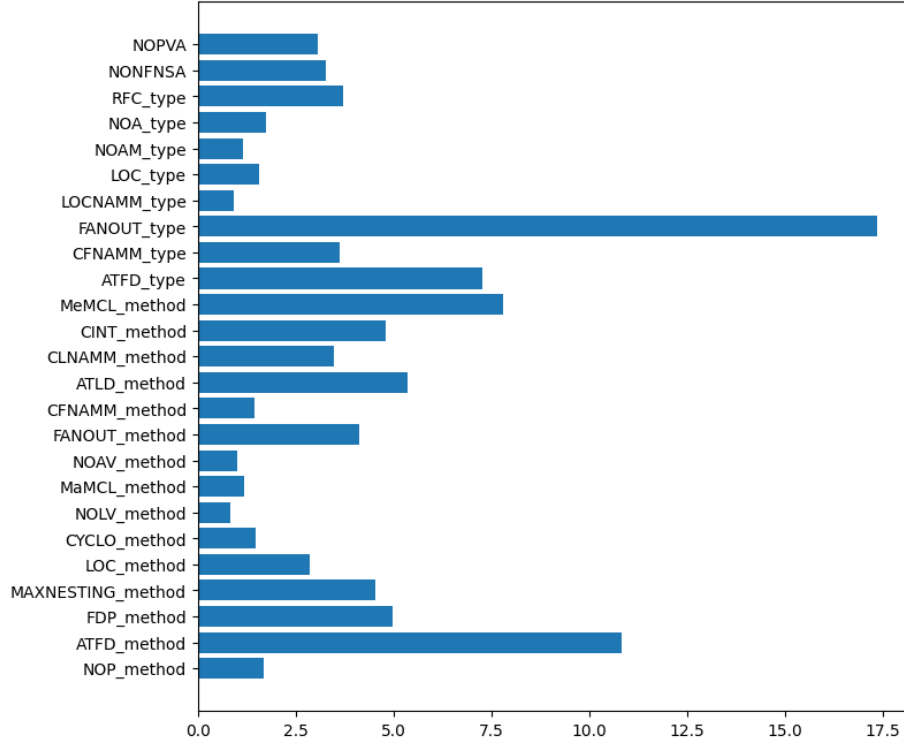


Figura 20 – Feature Importance for GC Severity Assessment in Java

D.2 SHAP

Figura 21 presents the SHAP summary plot showing the impact of features on the CB model output for GC detection. WMC_type, WMCNAMM_type, and LOCNAMM_type were the features with the greatest impact on the models output. In this regard, lower SHAP values for these features generally negatively influence the model’s output, while higher values tend to positively influence the output.

D.3 LIME

For the GC severity assessment in Java projects, a model based on CB without data scaling was used. Figura 22 shows the LIME graph for a non-severe GC instance, which in this case has a 90% probability of being correctly classified by the severity assessment model. This instance has the following attribute values: $MAXNESTING_method \leq 3.00$, $ATFD_type \leq 68.50$, $FANOUT_method \leq 0.00$, $NOLV_method \leq 1.00$, and $CLNAMM_method \leq 2.74$, which contribute to achieving this high probability.

For the GC severity instance shown in Figura 23, the prediction probability is 99%, supported by feature values such as: $MAXNESTING_method \leq 3.00$ and $FANOUT_method \leq 0.00$. On the other hand, $ATFD_type \leq 68.50$ and $LOCNAMM_type > 0.00$ are factors that reduce the prediction probability.

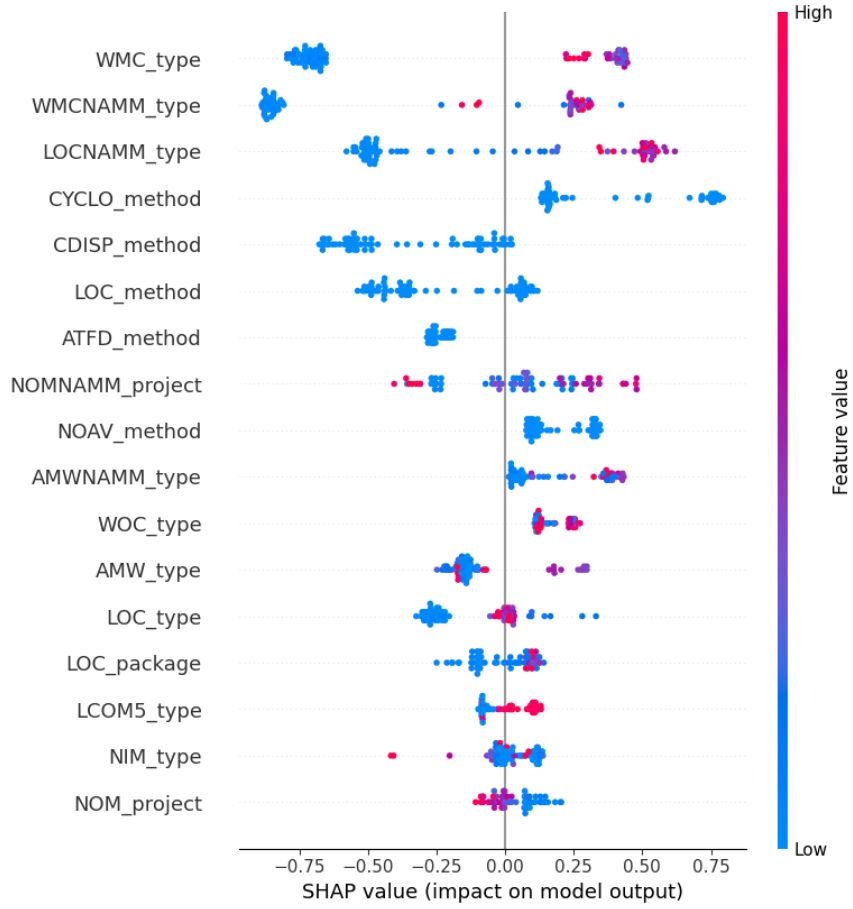


Figura 21 – Impact on the Output of the GC Detection Model for Java SHAP

In the case of a severe GC instance, detailed in Figura 24, the values *MAXNESTING_method* 3.00 and *LOC_type* > 3.00 are characteristics that increase the likelihood of correct classification. In this particular case, the prediction probability is 99%. This prediction probability was negatively affected by the values of some features (e.g., *ATFD_type* $\leq$ 68.50 and *CLNAMM_method* $\leq$ 2.74).



Figure 22 – LIME Explanation of a Non-Severe GC Instance in Java

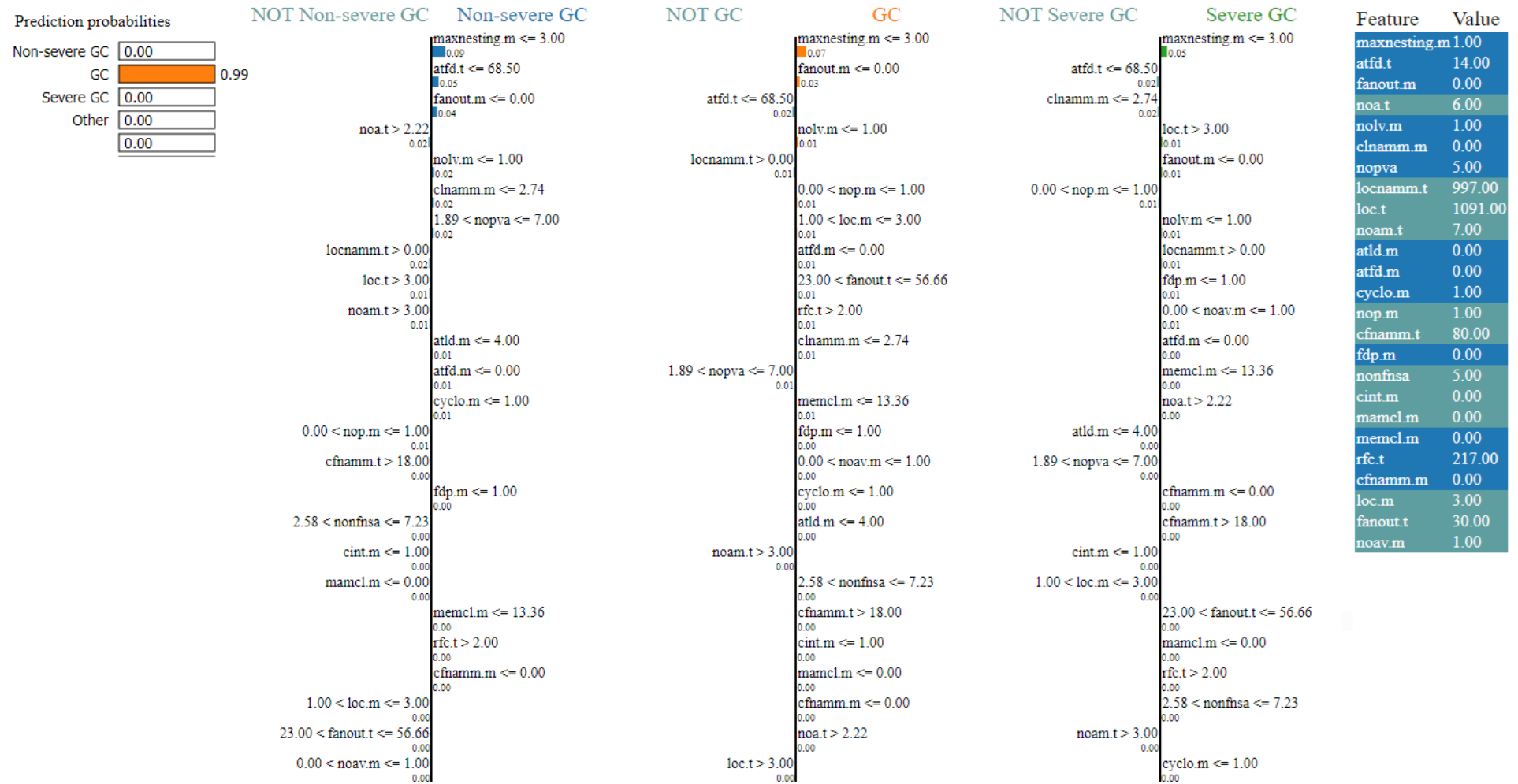


Figura 23 – LIME Explanation of a GC Severity Instance in Java

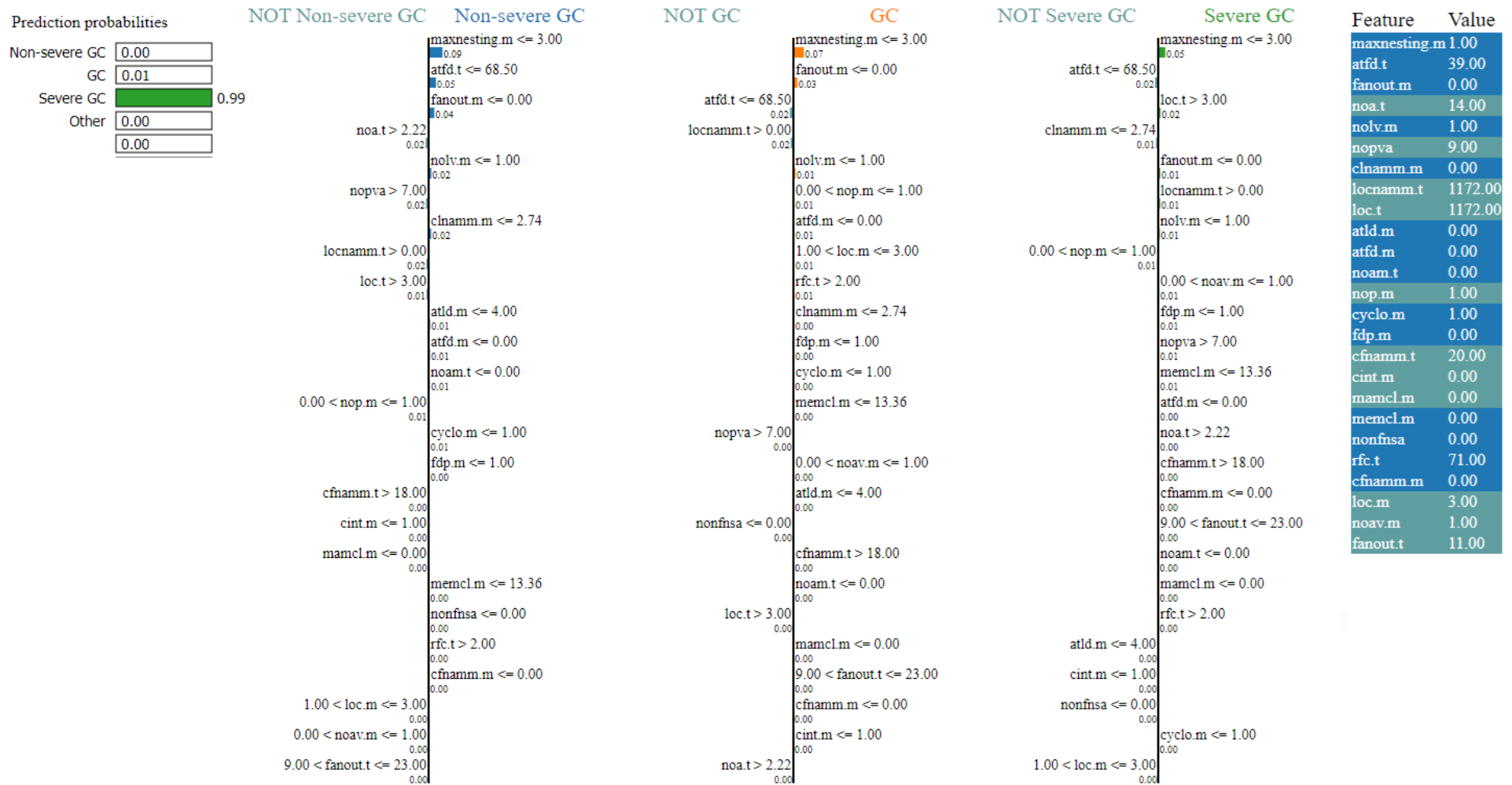


Figura 24 – LIME Explanation of a Severe GC Instance in Java